



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

INDEX

imports	3
STEP 1: Dataset Definition	3
STEP 2: Data Structure	4
STEP 3: Utility Functions.....	6
STEP 4: Adjusted Information Gain.	8
STEP 5: Splitting with Quinlan method	10
STEP 6: Recursive Tree Building	13
STEP 7: Tree Visualization	16
STEP 8: Main Execution.....	16
Analysis and Result	17

```
import math
```

از این ابزار برای محاسبه لگاریتم در آنتروپی استفاده شده است.

```
from collections import defaultdict
```

`defaultdict` یک نوع دیکشنری هوشمند است که اگر کلید وجود نداشته باشد، به طور خودکار مقدار اولیه می سازد.

```
import matplotlib.pyplot as plt
```

این کتابخانه برای و نمایش گرافیکی درخت استفاده شده است.

STEP 1: Dataset Definition

```
DATA = [
    [2, 2, 1, 0, 2],
    [2, 0, 1, 1, 0],
    [0, 0, 2, 0, 0],
    [1, 2, 0, 1, 2],
    [0, 2, None, 0, 0],
    [1, 2, 1, 0, 1],
    ...
    [1, 0, 0, 1, 1],
    [0, 2, 2, 1, 1]]
```

```
FEATURE_NAMES = ["f1", "f2", "f3", "f4"]
```

مجموعه داده شامل ۲۰ نمونه است. هر نمونه دارای چهار ویژگی گسسته `f1` تا `f4` با مقادیر ۰، ۱، ۲ و یک برچسب خروجی `y` با همین دامنه است. مقدار `None` نشان دهنده داده های گمشده^۱ در برخی ویژگی ها است. این ساختار داده برای آموزش درخت تصمیم، مورد استفاده قرار می گیرد.

¹ Missing Values

کلاس sample

```
class Sample:
```

یک کلاس به نام Sample تعریف شده که برای نمایش هر نمونه آموزشی در مجموعه داده استفاده می شود.

```
def __init__(self, features, label, weight=1.0):
```

این خط سازنده کلاس است که هنگام ایجاد یک آبجکت از نمونه اجرا می شود و سه ورودی می گیرد: ویژگی ها^۲، برچسب کلاس^۳ و وزن نمونه^۴ که مقدار پیش فرض آن ۱.۰ است.

```
self.features = features
```

مقدار ویژگی های نمونه را در آبجکت ذخیره می شود.

```
self.label = label
```

برچسب (کلاس خروجی) نمونه در آبجکت ذخیره می شود.

```
self.weight = weight
```

وزن نمونه ذخیره می شود (برای مدیریت داده های ناقص و اهمیت دهی به نمونه ها استفاده می شود).

کلاس DecisionNode

این کلاس برای گره های تصمیم گیری استفاده می شود؛ یعنی گره هایی که بر اساس مقدار یک ویژگی، داده ها را به شاخه های مختلف تقسیم می کنند.

```
class DecisionNode:
```

این خط یک کلاس با نام DecisionNode تعریف می کند که نمایانگر یک گره داخلی در درخت تصمیم است.

```
def __init__(self, feature_index):
```

این خط، سازنده کلاس است که شماره یا اندیس یکی از ۴ ویژگی مورد استفاده برای تقسیم را به عنوان ورودی دریافت می کند.

```
self.feature_index = feature_index
```

اندیس ویژگی انتخاب شده برای انجام تصمیم گیری در این گره ذخیره می شود.

```
self.children = {}
```

یک دیکشنری خالی ایجاد شده که شاخه های فرزند گره را نگهداری می کند. کلیدها مقادیر ممکن ویژگی و مقادیر آنها زیردرخت های متناظر هستند.

```
self.branch_probs = {}
```

یک دیکشنری برای ذخیره احتمال هر شاخه ایجاد شده است. این احتمالات در زمان مواجهه با مقادیر گمشده جهت توزیع نمونه ها بین شاخه های مختلف استفاده می شوند.

```
def is_leaf(self):
```

```
return False
```

تابعی برای تشخیص نوع گره تعریف شده است و مقدار False را برمی گرداند که نشان می دهد این گره یک گره برگ^۵ نیست، بلکه یک گره تصمیم گیری داخلی است که دارای شاخه های فرزند می باشد.

^۲ features

^۳ label

^۴ weights

^۵ Leaf

```
class LeafNode:
```

این کلاس گره‌های برگ در درخت تصمیم را مدل‌سازی می‌کند. گره برگ آخرین گره در مسیر تصمیم‌گیری بوده و نتیجه نهایی طبقه‌بندی را نگهداری می‌کند.

```
def __init__(self, label):
```

این خط سازنده کلاس است که برچسب کلاس نهایی را به عنوان ورودی دریافت می‌کند.

```
self.label = label
```

مقدار کلاس پیش‌بینی شده در گره برگ، ذخیره می‌شود.

```
def is_leaf(self):  
    return True
```

تابع `is_leaf` که برای تشخیص نوع گره تعریف شده است، مقدار `True` را برمی‌گرداند تا مشخص شود این گره یک گره برگ بوده و فرآیند تصمیم‌گیری در این نقطه به پایان رسیده است.

نکته: تابع `is_leaf()` در کلاس‌های `LeafNode` و `DecisionNode` با استفاده از پلی‌مورفیسم⁶ امکان تشخیص نوع گره را بدون نیاز به بررسی مستقیم کلاس شیء فراهم می‌کنند.

⁶ Polymorphism

تابع log 

```
def log(depth, message):
    indent = " " * depth
    print(f"{indent}{message}")
```

این تابع برای نمایش پیام‌های گزارش در طول اجرای برنامه تعریف شده است. ورودی `depth` نشان‌دهنده عمق بازگشت⁷ و `message` متن پیام مورد نظر است.

تابع weighted_majority_class 

```
def weighted_majority_class(samples):
```

این تابع کلاس غالب⁸ را بر اساس مجموع وزن نمونه‌ها محاسبه می‌کند.

```
    counts = defaultdict(float)
```

این خط، یک دیکشنری از نوع `defaultdict` ایجاد می‌کند که مقادیر اولیه آن از نوع اعشاری هستند. این دیکشنری برای نگهداری مجموع وزن هر کلاس استفاده می‌شود.

```
    for s in samples:
```

روی تمام نمونه‌های موجود در مجموعه داده پیمایش انجام می‌شود.

```
        counts[s.label] += s.weight
```

وزن هر نمونه را به مجموع وزن کلاس متناظر آن اضافه می‌شود.

```
    return max(counts.items(), key=lambda x: x[1])[0]
```

سپس تابع، کلاسی را که بیشترین مجموع وزن را دارد پیدا کرده و برمی‌گرداند. در اینجا `max` بر اساس مقدار وزن (عنصر دوم هر زوج کلید، مقدار) عمل می‌کند.

⁷ Recursion Depth

⁸ Majority Class

تابع `weighted_entropy`

این تابع آنتروپی شانن⁹ را محاسبه می‌کند، با این تفاوت که به جای شمارش ساده‌ی نمونه‌ها، از وزن نمونه‌ها استفاده می‌کند.

```
def weighted_entropy(samples):
```

ورودی آن `samples` است که مجموعه‌ای از نمونه‌هاست .

```
    total_weight = sum(s.weight for s in samples)
```

مجموع وزن تمام نمونه‌ها محاسبه می‌شود .

```
    if total_weight == 0:
        return 0.0
```

اگر هیچ وزنی وجود نداشته باشد (مثلاً همه وزن‌ها صفر شده باشند)، آنتروپی را صفر برمی‌گرداند. (برای جلوگیری از تقسیم بر ۰)

```
    class_weights = defaultdict(float)
```

یک دیکشنری ساخته می‌شود تا مجموع وزن هر کلاس ذخیره شود.

```
    for s in samples:
        class_weights[s.label] += s.weight
```

در این حلقه، برای هر نمونه، برچسبش را می‌گیرد و وزنش را به مجموع وزن آن کلاس اضافه می‌کند.

```
    ent = 0.0
```

متغیر آنتروپی، با صفر مقداردهی می‌شود .

```
    for w in class_weights.values():
```

این حلقه روی وزن کل هر کلاس پیمایش می‌شود .

```
        p = w / total_weight
```

احتمال وزن‌دار هر کلاس محاسبه می‌شود .
$$p_i = \frac{\text{total weight of samples in class } i}{\text{total weight of all samples}}$$

```
        if p > 0:
```

بررسی می‌شود احتمال صفر نباشد (چون $\log(0)$ تعریف نشده است).

```
            ent -= p * math.log2(p)
        return ent
```

جمله مربوط به آنتروپی اضافه می‌شود و در نهایت مقدار نهایی آنتروپی برمی‌گردانده می‌شود.

⁹ Shannon Entropy

STEP 4: Adjusted Information Gain

این تابع Information Gain را برای داده‌های ناقص را محاسبه می‌کند. ایده از روش Quinlan الگوریتم C4.5 گرفته شده است: وقتی بعضی نمونه‌ها مقدار ویژگی را ندارند، فقط روی نمونه‌های دارای مقدار آنتروپی را حساب می‌کنیم و در انتها نتیجه را با نسبت داده‌های شناخته‌شده تعدیل می‌کنیم.

$$\frac{|S - \sigma_{a_i=?} S|}{|S|} \Delta \Phi(a_i, S - \sigma_{a_i=?} S)$$

```
def adjusted_information_gain(samples, feature_index, depth=0):
```

```
    known = [s for s in samples if s.features[feature_index] is not None]
```

نمونه‌هایی که مقدار این ویژگی را دارند جدا می‌شوند.

```
    if len(known) == 0:
        return -1
```

اگر هیچ نمونه‌ای مقدار این ویژگی را نداشته باشد، این ویژگی قابل استفاده نیست و تابع 1-برمی‌گرداند.

```
    total_weight = sum(s.weight for s in samples)
```

وزن کل همه نمونه‌ها محاسبه می‌شود.

```
    known_weight = sum(s.weight for s in known)
```

وزن نمونه‌هایی که مقدار ویژگی مشخص دارند محاسبه می‌شود.

```
    parent_entropy = weighted_entropy(known)
```

آنتروپی مجموعه شناخته‌شده‌ها محاسبه می‌شود، نه کل داده‌ها.

```
    groups = defaultdict(list)
```

دیکشنری‌ای برای گروه‌بندی نمونه‌ها بر اساس مقدار ویژگی ساخته می‌شود.

```
    for s in known:
```

این حلقه روی نمونه‌های شناخته‌شده پیمایش می‌کند.

```
        groups[s.features[feature_index]].append(s)
```


نمونه داخل گروه مربوط به مقدار ویژگی قرار داده میشود.

```
remainder = 0.0
```

در این خط، باقیمانده آنتروپی بعد از تقسیم ویژگی، مقدار دهی اولیه میشود.

```
for value, subset in groups.items():
```

این حلقه روی هر مقدار ویژگی پیمایش می کند.

```
subset_weight = sum(x.weight for x in subset)
```

وزن کل آن زیرمجموعه محاسبه میشود.

```
subset_entropy = weighted_entropy(subset)
```

آنتروپی همان زیرمجموعه محاسبه میشود

```
remainder += (subset_weight / known_weight) * subset_entropy
```

میانگین وزن دار آنتروپی زیرمجموعه ها به remainder اضافه می شود.

```
gain = parent_entropy - remainder
```

Information Gain معمولی (فقط روی داده های شناخته شده) محاسبه میشود.

```
adjustment_factor = known_weight / total_weight
```

نسبت داده های قابل استفاده به کل داده ها در adjustment_factor ذخیره میشود.

```
adjusted_gain = adjustment_factor * gain
```

Gain نهایی تعدیل می شود تا ویژگی هایی که مقدارهای زیادی ندارند جریمه شوند.

```
return adjusted_gain
```

Gain تعدیل شده برگردانده می شود.

STEP 5: Splitting with Quinlan method

این تابع یکی از مهم ترین قسمت های برخورد با داده های ناقص در درخت تصمیم است. ایده اش این است که به جای حذف نمونه های ناقص، آن ها را به صورت احتمالی بین شاخه ها پخش کند. این دقیقاً سبک C4.5 است.

$$|\sigma_{a_i=v_{i,j}} S| / |S - \sigma_{a_i=?} S|$$

```
def split_with_missing(samples, feature_index, depth=0):
```

هدف تابع: تقسیم داده بر اساس یک ویژگی، با در نظر گرفتن مقادیر گمشده. این تابع سه ورودی دارد:

- Samples: مجموعه نمونه ها
- feature_index: اندیس ویژگی مورد استفاده برای split
- depth: عمق درخت برای logging

```
known = [s for s in samples if s.features[feature_index] is not None]
missing = [s for s in samples if s.features[feature_index] is None]
```

در این بخش، جداسازی داده های کامل و ناقص انجام میشود.

در این مرحله:

- Known: نمونه هایی که مقدار ویژگی را دارند
- missing: نمونه هایی که مقدار ویژگی را ندارند

```
branches = defaultdict(list)
```

یک ساختار داده ای برای نگهداری شاخه ها ایجاد می شود که کلید آن مقدار ویژگی و مقدار آن لیست نمونه ها است.

```
for s in known:
    branches[s.features[feature_index]].append(s)
```

هر نمونه بر اساس مقدار ویژگی خود به شاخه مربوطه اضافه می شود.

```
known_weight = sum(s.weight for s in known)
```

در این خط، وزن کل داده های معلوم محاسبه میشود.

$$W_{known} = \sum_{x \in S_{known}} w_x$$

```
branch_probs = {}
for value, subset in branches.items():
    branch_weight = sum(x.weight for x in subset)
    branch_probs[value] = branch_weight / known_weight
```

احتمال شاخه‌ها محاسبه میشود. احتمال هر شاخه بر اساس وزن تعریف می‌شود:

$$P(v) = \frac{W_v}{W_{known}}$$

```
for sample in missing:
```

در این حلقه، هر نمونه‌ای که مقدار ویژگی موردنظر را ندارد پردازش می‌شود.

```
for value, prob in branch_probs.items():
```

پیمایش روی شاخه‌ها انجام میشود. در اینجا الگوریتم روی همه شاخه‌های ممکن (values) حرکت می‌کند.

هر شاخه دارای:

- value: مقدار ویژگی
- Prob: احتمال تعلق نمونه به آن شاخه

```
copied = Sample(
    features=sample.features,
    label=sample.label,
    weight=sample.weight * prob)
```

در این بخش، نسخه وزن‌دار از نمونه ساخته میشود. اینجا مهم‌ترین بخش الگوریتم است. نمونه اصلی به چند نسخه تبدیل می‌شود، هر نسخه برای یک شاخه.

$$w' = w \cdot P(v)$$

وزن جدید:

```
branches[value].append(copied)
```

در این خط، نسخه‌ی وزن‌دار نمونه (copied) به شاخه‌ی مربوط به مقدار value اضافه می‌شود.

```
return branches, branch_probs
```

این خط دو خروجی نهایی الگوریتم را برمی‌گرداند:

۱. Branches

یک ساختار داده شامل:

- نمونه‌های واقعی (known data)
- نسخه‌های وزن‌دار از داده‌های missing

۲. branch_probs

$$P(v) = \frac{W_v}{W_{known}}$$

توزیع احتمال شاخه‌ها:

STEP 6: Recursive Tree Building

این تابع هسته‌ی اصلی ساخت درخت تصمیم است این تابع یک مدل بازگشتی می‌سازد که:

1. با آنروپی تصمیم می‌گیرد

2. با Information Gain ویژگی انتخاب می‌کند

3. با split احتمالاتی missing data را مدیریت می‌کند

4. با شرایط توقف از overfitting جلوگیری می‌کند

```
def build_tree(samples, available_features, depth=0):
```

ورودی‌ها:

- samples: داده‌های فعلی گره
- available_features: ویژگی‌های قابل استفاده
- depth: عمق درخت برای logging

```
labels = set(s.label for s in samples)
```

این خط:

- روی تمام samples پیمایش می‌کند
- برچسب هر نمونه را استخراج می‌کند
- سپس همه برچسب‌های تکراری را حذف می‌کند
- در نهایت یک مجموعه از کلاس‌های موجود در گره فعلی می‌سازد.

```
if len(labels) == 1:  
    label = next(iter(labels))  
    return LeafNode(label)
```

Stopping condition شماره یک: شرط خلوص

اگر همه نمونه‌ها متعلق به یک کلاس باشند:

- آنروپی = 0
- گره کاملاً خالص است
- بنابراین گره تبدیل به Leaf می‌شود.

```
if len(available_features) == 0:
    majority = weighted_majority_class(samples)
    return LeafNode(majority)
```

Stopping condition شماره دو: شرط نبود ویژگی

اگر دیگر ویژگی برای تقسیم وجود نداشته باشد:

- امکان split نیست
- باید تصمیم نهایی گرفته شود
- کلاس غالب (با وزن بیشتر) انتخاب می شود.

```
gains = []
for f in available_features:
    gain = adjusted_information_gain(samples, f, depth + 1)
    gains.append((gain, f))
```

در این بخش، Information Gain برای همه ویژگی ها محاسبه شده و سپس در لیست ذخیره میشود.

```
gains.sort(reverse=True)
best_gain, best_feature = gains[0]
```

ویژگی با بیشترین Information Gain انتخاب می شود.

```
If best_gain <= 0:
    majority = weighted_majority_class(samples)
    return LeafNode(majority)
```

Stopping condition شماره سه: شرط توقف بر اساس عدم بهبود

اگر هیچ ویژگی ای اطلاعات جدید اضافه نکند:

- split بی فایده است
- گره تبدیل به leaf می شود

```
node = DecisionNode(best_feature)
```

ایجاد یک گره داخلی در درخت:

- این گره بر اساس بهترین ویژگی split می کند

```
branches, probs = split_with_missing(samples, best_feature, depth + 1)
```

در این خط، تقسیم داده با مدیریت missing انجام میشود

این مهم ترین بخش است:

- داده ها بر اساس ویژگی انتخاب شده تقسیم می شوند
- داده های missing به صورت احتمالی پخش می شوند

```
node.branch_probs = probs
```

درخت اطلاعات توزیع شرطی شاخه ها را نگه می دارد

```
remaining_features = [f for f in available_features if f != best_feature]
```

جلوگیری از استفاده دوباره از همان ویژگی

```
for value, subset in branches.items():
```

در این خط، درخت به صورت بازگشتی ساخته میشود. برای هر مقدار ویژگی:

- یک زیرمسئله ساخته می شود
- درخت به صورت بازگشتی رشد می کند

```
node.children[value] = build_tree(  
    subset, remaining_features, depth + 1)
```

این همان تعریف بازگشتی درخت تصمیم است

```
return node
```

در نهایت یک درخت کامل یا subtree برگردانده می شود.

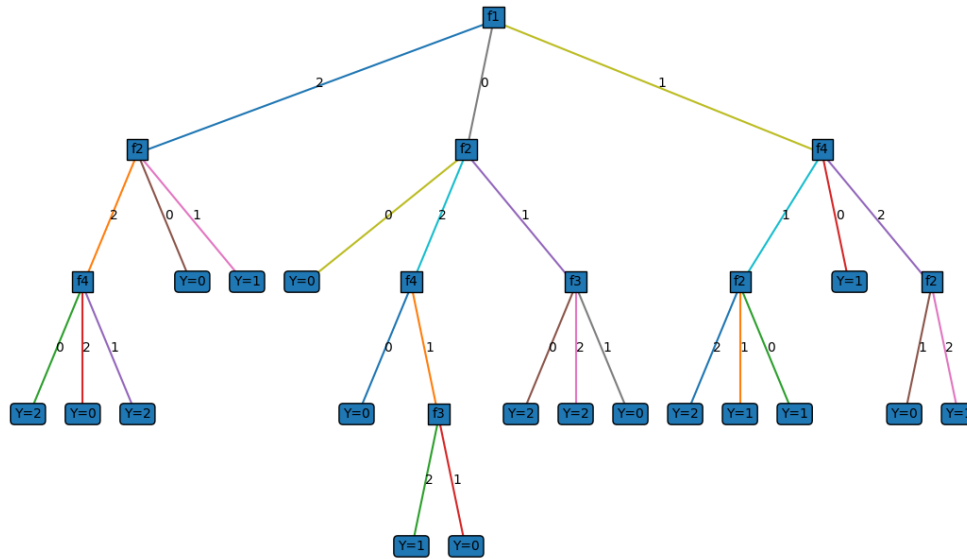
STEP 7: Tree Visualization

تابع `count_leaves` به صورت بازگشتی تعداد برگ‌های یک درخت تصمیم را محاسبه می‌کند؛ اگر گره فعلی یک برگ باشد مقدار ۱ برمی‌گرداند و در غیر این صورت تعداد برگ‌های تمام فرزندان را جمع می‌زند. این تابع برای تعیین ساختار و اندازه نسبی زیردرخت‌ها در فرآیند رسم استفاده می‌شود. تابع `draw_node` مسئول رسم بازگشتی درخت با استفاده از `matplotlib` است؛ اگر گره برگ باشد مقدار کلاس را در مختصات مشخص شده نمایش می‌دهد، و اگر گره داخلی باشد نام ویژگی تقسیم‌کننده را رسم کرده و سپس با استفاده از تعداد برگ‌های هر زیردرخت، موقعیت افقی و عمودی فرزندان را به صورت متناسب محاسبه می‌کند تا چیدمان درخت متعادل و بدون هم‌پوشانی باشد. در نهایت، خطوط اتصال بین گره‌ها و برچسب مقدار هر شاخه نیز رسم می‌شود و تابع به صورت بازگشتی برای تمام فرزندان ادامه پیدا می‌کند. تابع `draw_tree` نیز به عنوان نقطه شروع، یک بوم `matplotlib` ایجاد کرده، محورها را مخفی می‌کند، و سپس با فراخوانی `draw_node` کل درخت را از ریشه رسم کرده و در پایان تصویر را ذخیره و نمایش می‌دهد.

STEP 8: Main Execution

تابع `run_decision_tree_process` به عنوان یک تابع مدیریتی^{۱۰} کل فرایند ساخت درخت تصمیم را از ابتدا تا انتها اجرا می‌کند. ابتدا یک عنوان و توضیحات مربوط به روش مورد استفاده کاهش آنتروپی و مدیریت مقادیر گم‌شده با روش `Quinlan` را چاپ می‌کند تا روند اجرا شفاف شود. سپس داده‌های خام موجود در `DATA` را به آبجکت‌های `Sample` تبدیل می‌کند، به طوری که ویژگی‌ها در بخش ورودی و برچسب کلاس در بخش خروجی هر نمونه قرار می‌گیرد. در ادامه، تابع `build_tree` با استفاده از تمام ویژگی‌ها فراخوانی می‌شود تا درخت تصمیم به صورت بازگشتی ساخته شود. در نهایت، ساختار درخت تولیدشده با استفاده از تابع `draw_tree` رسم شده و در قالب یک تصویر در مسیر مشخص ذخیره می‌گردد.

¹⁰ wrapper



در این تمرین، یک درخت تصمیم مبتنی بر کاهش آنتروپی (Entropy Reduction) و معیار Information Gain تعدیل شده پیاده سازی شده که قادر به مدیریت داده های ناقص با استفاده از روش Quinlan (C4.5-style) می باشد. نتایج اجرای الگوریتم بر روی مجموعه داده شامل ۲۰ نمونه نشان می دهد که فرآیند انتخاب ویژگی ها به صورت سلسله مراتبی و بر اساس بیشترین کاهش عدم قطعیت انجام می شود.

در گره ریشه (depth = 0)، مقدار آنتروپی اولیه داده ها برابر ۱.۵۸۱۳ محاسبه شد که نشان دهنده توزیع نسبتاً نامتوازن اما پر از عدم قطعیت بین کلاس ها است. در این مرحله، چهار ویژگی f1 تا f4 ارزیابی شدند. نتایج نشان داد که ویژگی f1 با Information Gain برابر ۰.۲۵۸۸۸ بهترین عملکرد را دارد و به عنوان اولین معیار تقسیم انتخاب شد. این انتخاب نشان می دهد که f1 بیشترین توانایی را در کاهش عدم قطعیت داده ها نسبت به سایر ویژگی ها دارد. همچنین، در این سطح مشاهده شد که داده های ناقص وجود ندارند (known weight = total weight)، بنابراین نیازی به تعدیل ناشی از missing data در این گام نبود.

در سطح دوم (depth = 1)، داده ها بر اساس f1 به سه شاخه تقسیم شدند ۰، ۱ و ۲. (در این سطح، رفتار داده ها به طور قابل توجهی وابسته به مقدار ویژگی انتخاب شده بود. برای مثال، در شاخه $f1 = 2$ ، ویژگی f2 با Information Gain بسیار بالا (۰.۹۱۸۳۰) انتخاب شد که نشان دهنده وجود ساختار بسیار منظم و قابل تفکیک در این زیرمجموعه است. در مقابل، برخی شاخه ها مانند $f1 = 0$ و $f1 = 1$ نیز به سرعت به گره های خالص (Pure Nodes) منتهی شدند که در آن ها آنتروپی برابر صفر بوده و نیازی به ادامه تقسیم وجود نداشت.

یکی از مهم ترین بخش های این نتایج، مشاهده اثر داده های ناقص در سطح دوم شاخه $f1 = 0$ است. در این حالت، known weight کمتر از total weight بوده و بخشی از داده ها مقدار ویژگی f2 را نداشتند. الگوریتم با استفاده از روش احتمالاتی Quinlan، این نمونه های گمشده را

بین شاخه‌ها توزیع کرد (برای هر شاخه $\text{branch probability} \approx 0.333$). این رویکرد باعث شد که داده‌های ناقص حذف نشوند و در عین حال اثر آن‌ها به صورت وزنی در محاسبات لحاظ شود، که منجر به کاهش bias در انتخاب ویژگی شد.

در سطوح عمیق‌تر ($\text{depth} \geq 2$)، مشاهده شد که مقدار آنتروپی به طور پیوسته کاهش یافته و بسیاری از گره‌ها به حالت خالص رسیده‌اند. به عنوان مثال، در چندین زیرشاخه، Information Gain برابر ۱ یا نزدیک به آن بوده که نشان‌دهنده جداسازی کامل کلاس‌ها توسط ویژگی انتخاب شده است. این موضوع بیانگر قدرت بالای ویژگی‌های انتخاب شده در تفکیک داده‌ها و همچنین عملکرد صحیح معیار انتخاب ویژگی است.

در مجموع، نتایج نشان می‌دهد که مدل توانسته است با استفاده از ترکیب آنتروپی وزن‌دار و Information Gain تعدیل شده، یک ساختار درختی پایدار و قابل تفسیر ایجاد کند. همچنین، استفاده از روش Quinlan برای مدیریت داده‌های ناقص باعث شده است که اطلاعات از دست نرود و درخت بتواند حتی در حضور عدم قطعیت داده‌ای، تصمیم‌گیری‌های منطقی و آماری انجام دهد.

به طور خلاصه، این پیاده‌سازی نشان می‌دهد که:

- ویژگی f_1 در سطح اول بیشترین قدرت تفکیک را داشته است.
- برخی شاخه‌ها به سرعت به گره‌های خالص رسیده‌اند.
- داده‌های ناقص به صورت احتمالاتی مدیریت شده‌اند.
- و درخت نهایی ساختاری عمیق، منظم و قابل تفسیر دارد.

این نتایج تأیید می‌کند که استفاده از روش‌های مبتنی بر آنتروپی و مدیریت احتمالاتی داده‌های ناقص، عملکرد درخت تصمیم را به طور قابل توجهی بهبود می‌دهد.